VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Saahya K S (1BM23CS368)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Saahya K S (1BM23CS368),** who is a bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of a Machine Learning (23CS6PCMAL) work prescribed for the said degree.

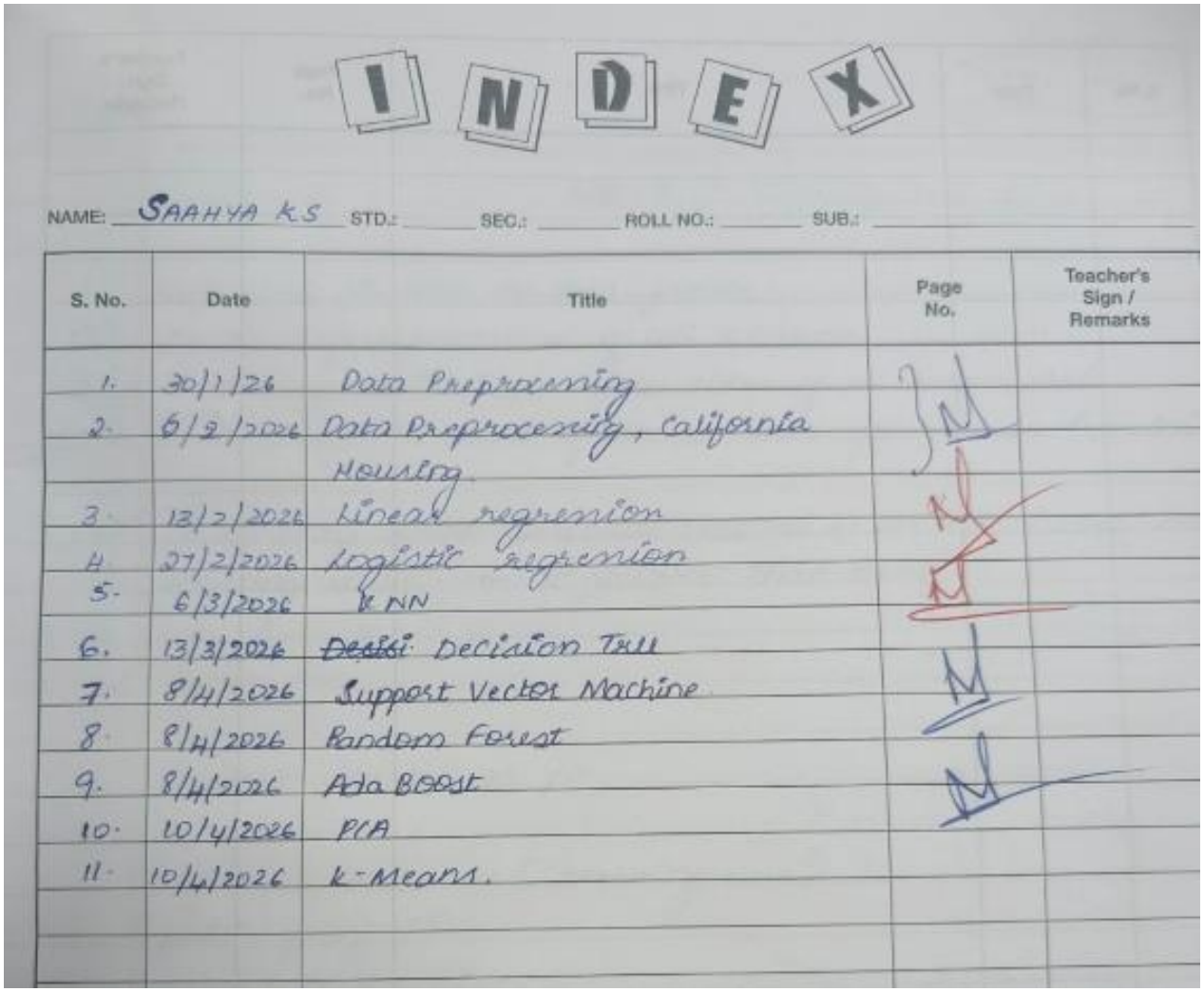
Megha J Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	4-2-2026	Write a python program to import and export data using Pandas library functions	5
2	11-2-2026	Demonstrate various data pre-processing techniques for a given dataset	9
3	25-2-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	15
4	25-3-2026	Build Logistic Regression Model for a given dataset	21
5	11-3-2025	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	25
6	25-2-2026	Build KNN Classification model for a given dataset.	31
7	1-4-2026	Build Support vector machine model for a given dataset	35
8	1-4-2026	Implement Random forest ensemble method on a given dataset.	38
9	1-4-2026	Implement Boosting ensemble method on a given dataset.	42
10	1-4-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	45
11	8-4-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	48

Github Link:

https://github.com/Saahya-KS17/ML-Lab



INDEX

NAME: SAAHYA K.S STD.: SEC.: ROLL NO.: SUB.:

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	30/1/26	Data Preprocessing		
2.	6/2/2026	Data Preprocessing, California Housing.		
3.	13/2/2026	Linear regression		
4.	27/2/2026	Logistic regression		
5.	6/3/2026	KNN		
6.	13/3/2026	~~Deci~~ Decision Tree		
7.	8/4/2026	Support Vector Machine		
8.	8/4/2026	Random Forest		
9.	8/4/2026	Ada BOOST		
10.	10/4/2026	PCA		
11.	10/4/2026	K-Means.		

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot

30/1/26.

LAB - 1

(i) load .csv file into the data frame
(ii) To display information of all columns
(iii) To display statistical information of all numerical
(iv) To display the count of unique labels for "Ocean Proximity" column
(v) To display which attributes (columns) in a dataset have missing values count greater than zero.

Python code.

```
import pandas as pd

df = pd.read_csv ("housing.csv")
print(df.info())
print ( df.describe())
print( df ['ocean_proximity'].value_counts())
print (df.isnull().sum() [df.isnull().sum() >0])
```

★ ~~After applying~~

★ Diabities dataset & Adult income dataset

1. Which columns in the dataset had missing values? How do did you handle them?

Ans) There were no missing values in ~~either of~~ the diabetes datasets but there was a 0 value in cholesterol column.

2. Categorical columns

(i) Diabetes dataset

~~No column in the dataset cont~~

Categorical columns identified were gender & class. These columns were encoded using label encoding ~~adult income datab set~~.

(ii) Adult Income dataset

The categorical columns identified were workclass, education, marital-status, occupation, relationship, race, sex, native country & income.

3. Min-max scaling vs standardization.

Minmax scaling scales data to a fixed range.

2. Categorical columns

(i) Diabetes dataset

~~No column in the dataset cont~~

Categorical columns identified were gender & class. These columns were encoded using label encoding ~~adult income datab set~~.

(ii) Adult Income dataset

The categorical columns identified were workclass, education, marital-status, occupation, relationship, race, sex, native country & income.

3. Min-max scaling vs standardization.

Minmax scaling scales data to a fixed range.

Code:

```
from sklearn.datasets import load_wine

import pandas as pd

import matplotlib.pyplot as plt
```

```
# Load dataset

wine = load_wine()

df = pd.DataFrame(wine.data, columns=wine.feature_names)

df['target'] = wine.target

print(df.head())

print(df.shape)

print(df.columns)

print(df.info())

print(df.describe())

print(df['target'].value_counts())

# Scatter plot

plt.scatter(df['alcohol'], df['malic_acid'])

plt.xlabel("Alcohol")

plt.ylabel("Malic Acid")

plt.show()

# Filter data

print(df[df['target'] == 0].head())

print(df[df['target'] == 1].head())

print(df.corr())

# Sort values

print(df.sort_values(by='alcohol').head())
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot

6/2/2026

LAB-2

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split,
StratifiedShuffleSplit
from sklearn.preprocessing import MinMaxScaler,
StandartScalar
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean-
absolute error, r2-score.
```

1. Dataset

```
url = "https://raw.githubusercontent.com/ageron/handson-ml,
    datasets/housing/housing.csv"
housing = pd.read_csv(url)
housing.head(10)
```

2. Basic statistics

```
housing.describe()
```

3. Histogram

```
plt.figure()
housing["median_house_value"].hist(bins=50)
plt.xlabel('Median House value')
plt.ylabel('Frequency')
plt.show()
```

4. ~~Bto~~ Box Plot

```
plt.figure()
sns.boxplot(x = housing['median_house_value'])
plt.show()
```

5&6. Identify Missing Values

```
percentage = housing.isnull().mean()*100
missing_percentage.
```

6 Handling missing values

```
imputer = SimpleImputer(strategy = 'median')
housing['total_bedrooms'] = imputer.fit_transform(
        housing[['total_bedrooms']])
```

7. Ocean_proximity

```
encode = pd.get_dummies(housing, columns = ['ocean_proximity
encode.head()
```

8. Feature Scaling (MinMax Scaling)

```
scaler = MinMaxScaler(feature_range = (0,1))
num_features = encode.drop('median_house_value', axis=1)
scaled = scaler.fit_transform(num_features)
scaled = pd.DataFrame(scaled, columns = num_features.columns
scaled.head()
```

9. Train-Test Split (80-20)

```
housing['~~ho~~ income_cat'] = pd.cut(
    housing['median_income'],
    bins = [0., 1.5, 3.0, 4.5, 6., np.inf],
    labels = [1, 2, 3, 4, 5])
```

```
split = StratifiedShuffleSplit(n_splits=1, test_size=0.2,
        random_state=42)
for train_idx, test_idx in split.split(housing, housing[
'income_cat']):
    strat_train_set = housing.loc[train_idx]
    strat_test_set = housing.loc[test_idx].
```

10. Correlation matrix

```
corr_matrix = housing.corr()
corr_matrix['median_housing_value'].sort_values(asc=Fals
```

Heatmap

```
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, cmap='coolwarm')
plt.show()
```

11. Scatter Plot

```
plt.figure()
plt.scatter( housing['median_income'],
             housing['median_house_value'],
             alpha=0.3)
plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.show()
```

12. Random Train-Test Split

```
train_set, test_set = Train_test_split
    housing, test_size=0.2, random_state=42)
```

13. Most Correlated Features

```
tar = corr_matrix['median_house_value'].sort_values(
    ascending=False)
top = tar[1:4]
neg = tar.tail(1)
top, neg
```

14. Standard Scaling

```
scale = StandardScaler()
num = housing.select_dtypes(include=[np.number]).columns
housing[num] = scale.fit_transform(housing[num_cols])
```

15. Linear Regression Model

```
X = housing.drop('median_house_value', axis=1)
y = housing['median_house_value']

X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
```

16. Model Evaluation

```
y_pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
rmse, mae, r2.
```

Code:

```
import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.datasets import load_diabetes

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression

from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Load dataset
diabetes = load_diabetes(as_frame=True)

# Create DataFrame
df = diabetes.frame
# Display basic info
print(df.head())
print(df.shape)
print(df.info())
# Summary statistics
print(df.describe())
# Check missing values
print(df.isnull().sum())
# Distribution plot
plt.figure()
sns.histplot(df['target'], bins=30, kde=True)
plt.xlabel("Target Value")
plt.ylabel("Frequency")
plt.title("Distribution of Target Values")
plt.show()

# Scatter plot
```

```
plt.figure()

sns.scatterplot(x=df['bmi'], y=df['target'], alpha=0.5)

plt.xlabel("BMI")

plt.ylabel("Target Value")

plt.title("BMI vs Target Value")

plt.show()
```

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot

13/2/2026

Date __/__/__
Page

Lab-3

Linear Regression & Multi-linear Regression

Q Find linear regression of the data of week & product sale

x_i (Week)	y_i (Sales in thousands)
1	2
2	4
3	5
4	9

$$\beta = ((X^T X)^{-1} X^T) Y$$

$$X^T X = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$$

$$(X^T X)^{-1} = \frac{1}{20} \begin{bmatrix} 4 & -10 \\ -10 & 30 \end{bmatrix} = \begin{bmatrix} 0.2 & -0.5 \\ -0.5 & 1.5 \end{bmatrix}$$

$$(X^T X)^{-1} X^T = \begin{bmatrix} 0.2 & -0.5 \\ -0.5 & 1.5 \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} =$$

$$= \begin{bmatrix} -0.3 & -0.8 & -1.3 & -1.8 \\ 1.0 & 2.5 & 4.0 & 5.5 \end{bmatrix}$$

$$((X^T X)^{-1} X^T) Y = \begin{bmatrix} -0.3 & -0.8 & -1.3 & -1.8 \\ 1.0 & 2.5 & 4.0 & 5.5 \end{bmatrix} \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$

$$\beta = \begin{bmatrix} -4.5 \\ 3.8 \end{bmatrix} \begin{matrix} \rightarrow \beta_0 \\ \rightarrow \beta_1 \end{matrix}$$

$$y = -4.5 + 3.8x$$

Date __/__/__
Page ____

1. Considering the 3 data.csv files, did you perform any data preprocessing steps

Ans) (i) In canada_per_capita_income.csv, no data preprocessing steps were applied

(ii) In salary.csv, no missing values so no preprocessing was done.

(iii) In hiring.csv we applied preprocessing
→ there were missing values in test_score, filled it with mean values.

2. Visualization of canada_per_capita_income.csv

Ans) Yes, the regression line was plotted along with the data points.

→ A strong positive linear relationship b/w Year & Per Capita Income.

→ As years increases, per capita income increase steadily.

→ The data points lie close to regression line, indicating a good linear fit.

3. Predicted Salary (hiring.csv)

Ans) Using the Multiple Linear Regression model:
For candidates with : 12 years experience, 10 Test score, 10 Interview score

The predicted salary is ~~[illegible]~~ ₹92268.07

4. 1000_companies.csv Preprocessing.

Ans) Yes, categorical variables were encoded
→ State col was converted into numerical using One-Hot Encoding with pd.get_dummies()

Date __/__/__
Page ____

1. Considering the 3 data.csv files, did you perform any data preprocessing steps

Ans) (i) In canada_per_capita_income.csv, no data preprocessing steps were applied

(ii) In salary.csv, no missing values so no preprocessing was done.

(iii) In hiring.csv we applied preprocessing
→ there were missing values in test_score, filled it with mean values.

2. Visualization of canada_per_capita_income.csv

Ans) Yes, the regression line was plotted along with the data points.

→ A strong positive linear relationship b/w Year & Per Capita Income.

→ As years increases, per capita income increase steadily.

→ The data points lie close to regression line, indicating a good linear fit.

3. Predicted Salary (hiring.csv)

Ans) Using the Multiple Linear Regression model:
For candidates with : 12 years experience, 10 Test score, 10 Interview score

The predicted salary is ~~[illegible]~~ ₹92268.07

4. 1000_companies.csv Preprocessing.

Ans) Yes, categorical variables were encoded
→ State col was converted into numerical using One-Hot Encoding with pd.get_dummies()

→ Scaling was not mandatory for linear regression. In this implementation, scaling was not applied because linear regression does not require it for correctness.

13/2/26

Code:

```
import pandas as pd

from sklearn.tree import DecisionTreeClassifier

from sklearn import tree

from sklearn.model_selection import train_test_split

from sklearn.datasets import load_wine, load_breast_cancer

import matplotlib.pyplot as plt

# Load Wine dataset

data = load_wine()

df = pd.DataFrame(data.data, columns=data.feature_names)

df["target"] = data.target

X = df.drop("target", axis=1)

y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42
```

```
)
model = DecisionTreeClassifier(criterion="entropy")
model.fit(X_train, y_train)

plt.figure(figsize=(18,10))
tree.plot_tree(
    model,
    feature_names=X.columns,
    class_names=data.target_names,
    filled=True
)
plt.show()

sample = [X.iloc[0].values]
prediction = model.predict(sample)

print("Wine Prediction:", data.target_names[prediction[0]])

# Load Breast Cancer dataset
data = load_breast_cancer()
df = pd.DataFrame(data.data, columns=data.feature_names)
df["target"] = data.target
X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(criterion="entropy")
model.fit(X_train, y_train)

plt.figure(figsize=(20,10))
tree.plot_tree(
    model,
    feature_names=X.columns,
    class_names=data.target_names,
    filled=True
)
plt.show()

sample = [X.iloc[0].values]
prediction = model.predict(sample)

print("Breast Cancer Prediction:", data.target_names[prediction[0]])
```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot

27/2/26

Lab - 4

Logistic Regression

1) Consider a binary classification problem where we want to predict whether a student will pass or fail based on their study hours. The logistic regression model has been trained, & the learned parameters are $a_0 = -5$ & $a_1 = 0.8$

(a) Write logistic regression eqn.

$y = a_1 x + a_0$

$\Rightarrow \boxed{y = 0.8x - 5}$

(b) Calculate probability that a student who studies for 7 hours will pass.

$$\text{sigmoid}(z) = \frac{1}{1+e^{-z}}$$

$z = 0.8(7) - 5 = 5.6 - 5 = 0.6$

$$\text{sigmoid}(z) = \frac{1}{1+e^{-0.6}} = 0.645$$

(c) Determine the predicted class (P/F), threshold of 0.5

Since, $0.645 > 0.5$

the student belongs to pass class.

2. Consider $z = [2, 1, 0]$ for three classes. Apply SoftMax function to find the probability values of 3 classes.

Sol) $\text{SoftMax} = \frac{e^{z_i}}{\sum_{i=1}^{k} e^{z}}$

$e^2 = 7.389$

$e^1 = 2.718$

$e^0 = 1$

$$\Sigma e^z = 7.389 + 2.718 + 1 = 11.107$$

Class 1:

$$\text{Softmax} = \frac{7.389}{11.107} = 0.665$$

Class 2:

$$\text{Softmax} = \frac{2.718}{11.107} = 0.2415$$

Class 3:

$$\text{Softmax} = \frac{1}{11.107} = 0.091$$

$$[0.665, 0.2415, 0.091]$$

Code:

```
import numpy as np

from sklearn.datasets import load_diabetes

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LinearRegression

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
# Load dataset
data = load_diabetes()
X = data.data
y = data.target
# Select features
selected_features = ['bmi', 'bp', 's1']

# Get feature indices
feature_names = data.feature_names
print("All features:", feature_names)

selected_indices = [feature_names.index(f) for f in selected_features]

# Extract selected features
X_selected = X[:, selected_indices]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y, test_size=0.2, random_state=42
)
# Standardization
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# Train Linear Regression model
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

# Predictions
y_pred_lr = lin_reg.predict(X_test_scaled)

# Evaluation
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

print("Selected features:", selected_features)
print("RMSE:", rmse_lr)
print("MAE:", mae_lr)
print("R2 Score:", r2_lr)
print("Coefficients:", lin_reg.coef_)
print("Intercept:", lin_reg.intercept_)
```

Program 5

Build Logistic Regression Model for a given dataset

Screenshot

6/3/26

Lab-5

K-Nearest Neighbour (KNN)

1) Consider the following dataset, for k=3 & test data (x, 35, 100) as (Person, Age, Salary k) solve using KNN classifier model & predict the ~~model~~ target

Person	Age	Salary k	Target	Distance	Rank
A	18	50	N	2517	5
B	23	55	N	2037	4
C	24	70	N	911	2
D	41	60	Y	1606	3
E	43	70	Y	908	1
F	38	40	Y	3603	6
X	35	100	Y		

∴ (x, 35, 100) → Yes (Y)

★ Iris dataset → Accuracy = 80%

1) How to choose the k value? Demonstrate using accuracy rate & error rate.

Ans) The value of k is chosen by testing different k values (1, 3, 5, 7, etc) & calculating the accuracy rate for each.

It is always better to choose k value to be an odd number.

The error rate is calculated as:

Error Rate = 1 − Accuracy.

The k value with highest accuracy & lowest error rate is ~~tett~~ selected as the optimal k.

★ 2) Diabetes dataset => Accuracy = 69.48%

2) What is the purpose of feature scaling? How to perform it?

Ans) Feature scaling is used to bring all features to the same range so that no feature dominates the distance calculation of KNN

Since KNN uses distance measure, features with large values can affect the result more if scaling is not applied.

Feature scaling performed using standardization

$$z = \frac{x - \text{mean}}{\text{standard deviation}}$$

This ~~data~~ makes the data mean = 0 & standard deviation = 1

Code:

```
from google.colab import files

uploaded=files.upload()

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

# Load dataset

df = pd.read_csv("HR_comma_sep.csv")

# Preview

print(df.head())

print(df.info())

print(df.describe())

print(df['left'].value_counts())

sns.countplot(x='left', data=df)

plt.title("Employee Retention Count (0 = Stayed, 1 = Left)")

plt.show()


corr = df.corr(numeric_only=True)

plt.figure(figsize=(10,6))

sns.heatmap(corr, annot=True, cmap='coolwarm')
```

```
plt.title("Correlation Heatmap")
plt.show()

salary_retention = df.groupby('salary')['left'].mean()

salary_retention.plot(kind='bar')
plt.title("Impact of Salary on Employee Retention")
plt.ylabel("Probability of Leaving")
plt.show()

dept_retention = df.groupby('Department')['left'].mean()

dept_retention.plot(kind='bar', figsize=(10,5))
plt.title("Department vs Employee Retention")
plt.ylabel("Probability of Leaving")
plt.xticks(rotation=45)
plt.show()

# One-hot encoding
df = pd.get_dummies(df, columns=['salary', 'Department'], drop_first=True)

X = df.drop('left', axis=1)
y = df['left']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Program 6

Build KNN Classification model for a given dataset.

Screenshot

13/3/2026

lab-6

Decision Tree

Consider the following dataset, calculate Entropy & information gain wrt target variable "Classification". Identify whether the splitting node should be a2 or a3 attribute.

Instance	a2	a3	Classification
1	Hot	High	No
2	Hot	High	No
6	Cool	High	No
7	Hot	High	No
8	Hot	Normal	Yes

Total samples = 5

Yes = 1

No = 4

$$H(S) = \frac{4}{5} \cdot \log_2\left(\frac{4}{5}\right) - \frac{1}{5} \log_2\left(\frac{1}{5}\right)$$

$$= 0.7219$$

Implementation of a2 (a2 = No)

Hot = 3 Cool = 1

$$H(a_2 = Hot) = \frac{-3}{4} \log_2 \frac{3}{4} - \frac{1}{4} \log_2 \frac{1}{4} = 0.843$$

$$H(a_2 = Cool) = \frac{0}{1} \log_2 \frac{0}{1} - \frac{1}{1} \log_2 \frac{1}{1} = 0$$

1) For 'Iris.csv' dataset
Accuracy score = 100%

What does confusion matrix tell you about the model's performance? Were there any misclassifications? If so, which classes were most confused?

Ans) The confusion matrix shows the no of correct & incorrect predictions made by model.

→ Most predictions lie on the diagonal, indicating correct classifications
→ Setosa flowers are usually classified perfectly because their features are very distinct.
→ A few misclassifications may occur b/w Versicolor & Virginica since their feature values are similar.

2) For 'petrol_consumption.csv' dataset
Can you interpret the ~~Deciision~~ Regression Tree structure? What are the most important features for predicting petrol consumption?

Ans) The Regression Tree predicts petrol consumption by splitting the dataset based on features such as
→ Average Income
→ Petrol price
→ Population with driving license
→ Highway miles.

The Tree selects the feature that the best reduce the prediction error @ each split.

Code:

```
import numpy as np

from sklearn.datasets import fetch_california_housing

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.neighbors import KNeighborsRegressor
```

```
# Load dataset
data = fetch_california_housing()
X = data.data
y = data.target

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Standardization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Train KNN Regressor
knn = KNeighborsRegressor(n_neighbors=3)
knn.fit(X_train, y_train)
```

```
# New data prediction
new_house = np.array([[8.3252, 41, 6.984, 1.023,
                       322, 2.555, 37.88, -122.23]])

new_house_scaled = scaler.transform(new_house)

predicted_price = knn.predict(new_house_scaled)

print("Predicted House Price (in $100,000s):", predicted_price[0])
print("Predicted Price in Dollars: $", predicted_price[0] * 100000)
```

Program 7

Build Support vector machine model for a given dataset.

Screenshot

8/4/2024

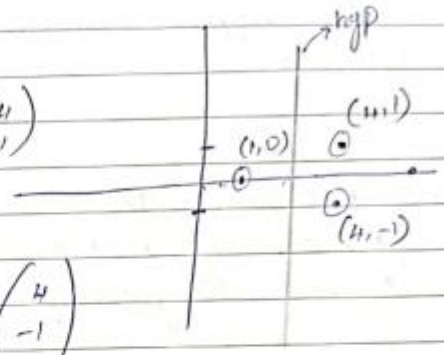
lab-7

Support Vector Machine

Q Points (4,1), (4,-1) & (6,0) belong to +ve class & points (1,0), (0,1) and (0,-1) belong to negative class. Draw an optimal hyperplane.

Sol) Support vectors are:

$S_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad S_2 = \begin{pmatrix} 4 \\ 1 \end{pmatrix}, \quad S_3 = \begin{pmatrix} 4 \\ -1 \end{pmatrix}$



Adding bias,

$S_1 = \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}, \quad S_2 = \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix}, \quad S_3 = \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix}$

Eqns:

$\alpha_1 S_1 S_1 + \alpha_2 S_2 S_1 + \alpha_3 S_3 S_1 = -1$

$\alpha_1 S_1 S_2 + \alpha_2 S_2 S_2 + \alpha_3 S_3 S_2 = 1$

$\alpha_1 S_1 S_3 + \alpha_2 S_2 S_3 + \alpha_3 S_3 S_3 = 1$

$$\alpha_1 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_2 \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + \alpha_3 \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} = -1$$

$2\alpha_1 + 5\alpha_2 + 5\alpha_3 = -1 \rightarrow (1)$

$5\alpha_1 + 18\alpha_2 + 16\alpha_3 = 1 \rightarrow (2)$

$5\alpha_1 + 16\alpha_2 + 18\alpha_3 = 1 \rightarrow (3)$

$\alpha_1 = -2.44$

$\alpha_2 = 0.38$

$\alpha_3 = 0.38$

$$W = \sum_{1}^{3} \alpha_i S_i = -2.44 \begin{pmatrix} 1 \\ 0 \\ 1 \end{pmatrix} + 0.38 \begin{pmatrix} 4 \\ 1 \\ 1 \end{pmatrix} + 0.388 \begin{pmatrix} 4 \\ -1 \\ 1 \end{pmatrix}$$

$$= \begin{pmatrix} -2.44 \\ 0 \\ -2.44 \end{pmatrix} + \begin{pmatrix} 1.52 \\ 0.38 \\ 0.38 \end{pmatrix} + \begin{pmatrix} 1.52 \\ -0.38 \\ 0.38 \end{pmatrix} = \begin{pmatrix} 0.6 \\ 0 \\ -1.68 \end{pmatrix}$$

SVM on letter-recognition.csv dataset

Q1. Some letters with similar shapes are confused:
* O & Q
* I & L
* C & G

This happens because their features are visually similar.

Q2. AUC score: 0.98
* AUC close to 1 → excellent model
* Shows strong classification ability across all classes.

Q3.

Dataset	Accuracy	Complexity
Iris	very high	simple.
letter-recognition	slightly slower	Complex.

→ SVM performs better on datasets like Iris.
→ Performance decreases slightly for complex-class data.

Code:

```
import numpy as np


from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler
```

```
from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, classification_report

from sklearn.datasets import load_digits


data = load_digits()

X = data.data

y = data.target

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = SVC(kernel='linear')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

new_digit = np.array([X[0]])

new_digit = scaler.transform(new_digit)

prediction = model.predict(new_digit)

print("\nPredicted Digit:", prediction[0])
```

Program 8

Implement Random forest ensemble method on a given dataset.

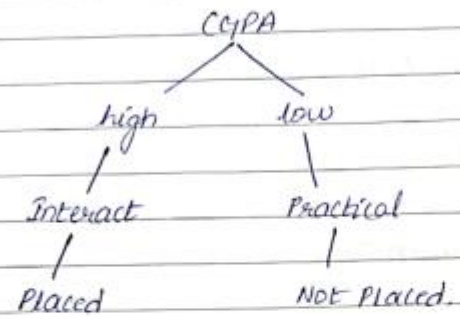
Screenshot

01/01/2026

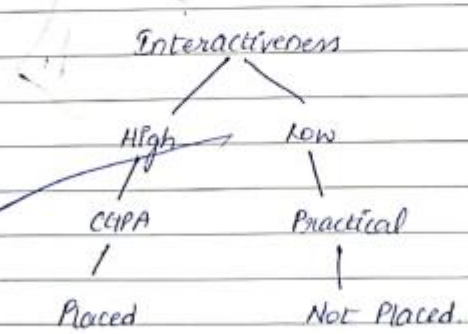
Lab - 8

Random Forest

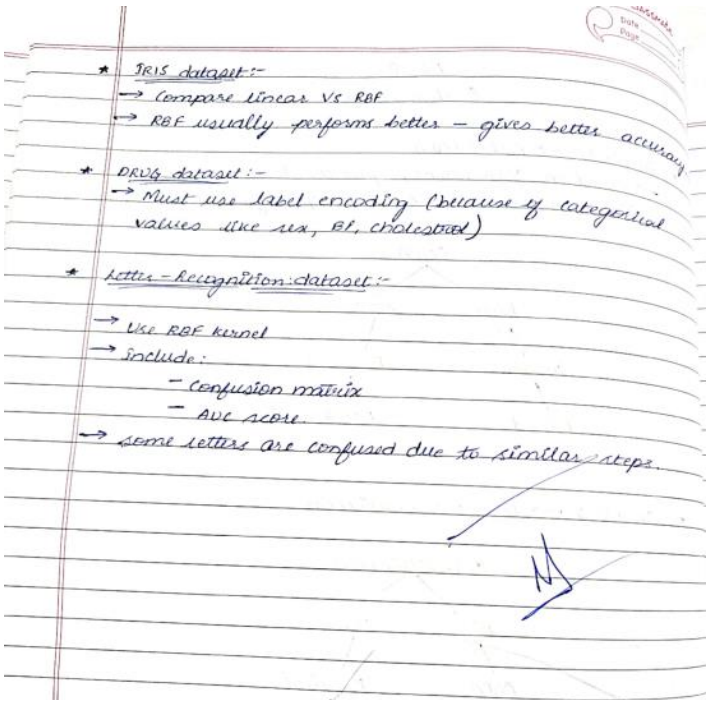
* Sample 1 → Root = CGPA
→ Based on CGPA values, split the data into branches.
→ Then use other attributes to split further.



* Sample 2 → Root = Interactiveness



Q1: Accuracy score is 0.97 => 97%.
Q2: Number of Trees used = 100
Q3: Confusion matrix $\begin{bmatrix} 14 & 0 & 1 \\ 0 & 13 & 1 \\ 0 & 1 & 16 \end{bmatrix}$



* IRIS dataset:-
  → Compare linear vs RBF
  → RBF usually performs better – gives better accuracy

* DRUG dataset:-
  → Must use label encoding (because of categorical values like sex, BP, cholesterol)

* Letter-Recognition dataset:-
  → Use RBF kernel
  → Include:
    – Confusion matrix
    – AUC score
  → some letters are confused due to similar steps.

Code:

```
import numpy as np

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load Breast Cancer dataset

from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()

X = data.data

y = data.target

print("Feature shape:", X.shape)

print("Target shape:", y.shape)

# Create DataFrame

df = pd.DataFrame(X, columns=data.feature_names)

df["target"] = y
```

```
# Save CSV
file_name = "random_forest_breast_cancer.csv"
df.to_csv(file_name, index=False)
from google.colab import files
files.download(file_name)
uploaded = files.upload()
# Load dataset
df = pd.read_csv("random_forest_breast_cancer.csv")

print(df.head())
print(df.info())

# Split features and target
X = df.iloc[:, :-1]
y = df.iloc[:, -1]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train Random Forest model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predictions
```

```
y_pred = model.predict(X_test)


# Evaluation

print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nClassification Report:\n", classification_report(y_test, y_pred))

print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot

Lab-9

Ada Boost

Q1: Accuracy score → 0.88 to 0.93

Number of trees used — 400 estimators

Confusion matrix → $\begin{bmatrix} TN & FP \\ FN & TP \end{bmatrix} = \begin{bmatrix} 3110 & 60 \\ 70 & 330 \end{bmatrix}$

Q2: It improves classification performance by combining multiple weak learners. Initially all data points are given equal importance. During training, misclassified points are given higher weight that subsequent models focus more on difficult cases.

As the number of estimators increases, the model becomes more accurate up to a certain limit. After that performance may stabilize or slightly decrease due to overfitting.

Q3. Interpretation of confusion.

→ True positive.

→ True negative

→ False positive

→ False negative.

Q4. Conclusion:

AdaBoost classifier performs well on the income dataset by improving weak learners iteratively. The tuned model with optimal number of provider better accuracy compared to the default model.

Code:

```
import pandas as pd

import numpy as np
```

```
from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score, confusion_matrix


import seaborn as sns

import matplotlib.pyplot as plt


# Load dataset

data = load_breast_cancer(as_frame=True)

df = data.frame


# Features and target

X = df.drop('target', axis=1)

y = df['target']


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


# Train AdaBoost model

model = AdaBoostClassifier(n_estimators=50, random_state=42)

model.fit(X_train, y_train)
```

```
# Predictions
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")

plt.show()
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot

10/11/2024

Lab - 10

PCA

Q. Given the data in table, reduce the dimension from 2 to 1 using PCA. Compute the First Principal Component.

Feature	Example 1	Example 2	Example 3	Example 4
x_1	4	8	13	7
x_2	11	4	5	14

Consider the following eigen values & vectors:

$\lambda_1 = 30.3847$ $\quad e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$, $\quad e_2 = \begin{bmatrix} 0.8303 \\ 0.5573 \end{bmatrix}$

$\lambda_2 = 6.6151$

Sol) Mean: $\bar{x}_1 = 8$

$\bar{x}_2 = 8.5$

Mean center: $x' = x - \bar{x}$

Ex	x_1'	x_2'	PC_1
1	-4	2.5	-4.30535
2	0	-4.5	3.73635
3	5	-3.5	5.69305
4	-1	5.5	-5.12405

$PC_1 = 0.5574 x_1' - 0.8303 x_2'$

Before PCA:

Logistic Regression: 0.8478

SVM: 0.8641

Random Forest: 0.8967

After PCA:

Logistic Regression: 0.8261

SVM: 0.8315

Random Forest: 0.7717

Conclusion: Accuracy decreased after PCA (trade-off for lower dimensionality)

Code:

```
from google.colab import files

uploaded = files.upload()
```

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler


# Load dataset

data = pd.read_csv("Mall_Customers.csv")

print(data.head())


# Select features

X = data[['Annual Income (k$)', 'Spending Score (1-100)']]


# Standardize features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Elbow method (WCSS)

wcss = []

for i in range(1, 11):

    kmeans = KMeans(n_clusters=i, random_state=42)

    kmeans.fit(X_scaled)  wcss.append(kmeans.inertia_


# Plot Elbow Graph

plt.plot(range(1, 11), wcss, marker='o')

plt.title("Elbow Method")

plt.xlabel("Number of Clusters")
```

```
plt.ylabel("WCSS")
plt.show()

# Apply KMeans (k=5)
kmeans = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans.fit_predict(X_scaled)

# Add cluster labels
data['Cluster'] = y_kmeans
print(data.head())

# Plot clusters
plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_kmeans, cmap='rainbow')
plt.title("Customer Segments")
plt.xlabel("Annual Income (scaled)")
plt.ylabel("Spending Score (scaled)")
plt.show()
```

Program 11

Implement Dimensionality reduction using the Principle Component Analysis (PCA) method.

Screenshot

10/11/2026

Lab-11

k-Means

Q. For the given data compute 2 clusters using k-means algo for clustering where initial cluster centers are (1.0, 1.0) & (5.0, 7.0)

Record No.	A	B	(A,B)	d_1	d_2	Cluster
R1	1.0	1	(1, 1)	0	7.21	C_1
R2	1.5	2	(1.5, 2)	1.12	6.10	C_1
R3	3.0	4	(3, 4)	3.61	3.61	C_1
R4	5.0	7	(5, 7)	7.21	0	C_2
R5	3.5	5	(3.5, 5)	4.72	2.5	C_2
R6	4.5	5	(4.5, 5)	5.32	2.06	C_2
R7	3.5	4	(3.5, 4)	4.3	2.72	C_2

New Centroid:

$$C_1 = \left(\frac{1+1.5+3}{3}, \frac{1+2+4}{3}\right) = (1.83, 2.33)$$

$$C_2 = (4.125, 5.375)$$

Records	d_1	d_2	Cluster
R1	1.57	5.38	C_1
R2	6.47	4.3	C_1
R3	2.06	1.79	C_2
R4	5.59	1.91	C_2
R5	3.17	0.73	C_2
R6	3.80	0.95	C_2
R7	2.71	1.06	C_2

New:

$C_1 = (1.25, 1.5)$

$C_2 = (3.9, 5.1)$

Q1: → From the elbow point plot, the curve shows a clear bend @ k = 3

→ Hence, the optimal no. of clusters is 3.

Code:

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.preprocessing import StandardScaler

from sklearn.decomposition import PCA

import matplotlib.pyplot as plt
```

```
iris = load_iris()

X = iris.data

y = iris.target

# Standardize features

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

# Apply PCA (2 components)

pca = PCA(n_components=2)

X_pca = pca.fit_transform(X_scaled)

# Explained variance

print("Explained Variance Ratio:", pca.explained_variance_ratio_)

print("Total Variance Retained:", sum(pca.explained_variance_ratio_))

# PCA components (feature contributions)

print("PCA Components:\n", pca.components_)

# Plot

plt.figure(figsize=(8, 6))

for i in range(3):

    plt.scatter(X_pca[y == i, 0], X_pca[y == i, 1], label=iris.target_names[i])

plt.xlabel("Principal Component 1")

plt.ylabel("Principal Component 2")

plt.legend()

plt.title("PCA on Iris Dataset")

plt.show()
```